Linux Network Programming — Hands-On Lab Manual

📋 Prerequisites

- A Linux machine (Ubuntu 20.04+ / Debian 11+) — native, VM, or Raspberry Pi
- Root/sudo access
- GCC, Make installed
- Basic C programming and Linux command-line familiarity
- Two machines (or two terminals on one machine) for client-server testing

```
# Quick setup — run this first!
sudo apt update
sudo apt install -y build-essential gcc make \
    net-tools iproute2 iputils-ping traceroute dnsutils \
    nmap netcat-openbsd curl wget \
    tcpdump wireshark tshark \
    iptables nftables \
    isc-dhcp-server bind9-utils ntp \
    linux-headers-$(uname -r) \
    libpcap-dev libnetfilter-queue-dev \
    python3 git
```

> 🍓 ***Raspberry Pi Users:*** *All user-space socket programming labs work on RPi. Kernel network module labs require* `raspberrypi-kernel-headers` *. Raw socket and netfilter labs require root. Wi-Fi driver exploration requires a wireless adapter.*

> ⚠️ ***Important:*** *Raw socket programs, netfilter modules, and kernel network code require root privileges and can disrupt network connectivity. Always test in a VM or on an isolated network.*

PART 1 — APPLICATION NETWORK PROGRAMMING

LAB 1: Introduction & Network Configuration

🎯 Mission: "Mapping the Network Terrain"

> ***Story:*** *Before writing any network code, you must understand the battlefield — your network interfaces, IP addresses, routing tables, and DNS configuration. A soldier who doesn't know the terrain is already lost.*

📖 Background

Every network program relies on the operating system's network stack:

```
┌─────────────────────────────────────────────────────┐
│                 Application (your code)             │
│                 socket(), bind(), listen(), ...     │
├─────────────────────────────────────────────────────┤
│             Socket API (POSIX / BSD Sockets)        │
├─────────────────────────────────────────────────────┤
│  Transport Layer:  TCP (SOCK_STREAM)  UDP (SOCK_DGRAM)  │
├─────────────────────────────────────────────────────┤
│  Network Layer:    IPv4 / IPv6                      │
```

```
├──────────────────────────────────────────────────────────┤
│  Data Link Layer:  Ethernet / Wi-Fi / Loopback           │
├──────────────────────────────────────────────────────────┤
│  Physical Layer:   NIC Hardware                          │
└──────────────────────────────────────────────────────────┘
```

OSI Model vs TCP/IP Model

```
   OSI Model              TCP/IP Model
┌────────────────┐       ┌────────────────┐
│ Application    │       │                │
├────────────────┤       │ Application    │
│ Presentation   │       │ (HTTP, DNS,    │
├────────────────┤       │  FTP, SSH)     │
│ Session        │       │                │
├────────────────┤       ├────────────────┤
│ Transport      │       │ Transport      │
│ (TCP, UDP)     │       │ (TCP, UDP)     │
├────────────────┤       ├────────────────┤
│ Network        │       │ Internet       │
│ (IP, ICMP)     │       │ (IP, ICMP)     │
├────────────────┤       ├────────────────┤
│ Data Link      │       │ Network        │
├────────────────┤       │ Access         │
│ Physical       │       │ (Ethernet)     │
└────────────────┘       └────────────────┘
```

🏁 Exercise 1.1: Network Interface Exploration

Objective: Discover your system's network configuration.

Step 1: List network interfaces:

```
# Modern iproute2 commands
ip addr show              # All interfaces with addresses
ip link show              # Link layer info
ip -s link show           # With statistics

# Legacy commands (still common)
ifconfig -a               # All interfaces
```

Step 2: Examine routing table:

```
ip route show             # Routing table
ip route get 8.8.8.8     # Route to specific destination
route -n                  # Legacy routing table

# Default gateway
ip route | grep default
```

Step 3: DNS configuration:

```
cat /etc/resolv.conf       # DNS resolver config
systemd-resolve --status   # systemd DNS (if applicable)
dig example.com            # DNS lookup
```

```
host example.com           # Simple DNS lookup
nslookup example.com       # Another DNS tool
```

Step 4: Network statistics:

```
ss -tuln                   # Listening sockets (TCP/UDP)
ss -tnp                    # Established TCP connections with PIDs
netstat -tuln              # Legacy equivalent

# ARP table
ip neigh show              # ARP cache
arp -a                     # Legacy ARP

# Connection tracking
cat /proc/net/tcp | head -5
cat /proc/net/udp | head -5
```

Step 5: Network testing:

```
ping -c 4 8.8.8.8                 # ICMP connectivity
traceroute example.com             # Path tracing
mtr example.com                    # Combined ping+traceroute
curl -v http://example.com 2>&1 | head -30   # HTTP test
```

Step 6: Answer in `answers.txt`:

1. What is your machine's IP address and subnet mask?
2. What is the default gateway?
3. What DNS server is configured?
4. How many listening TCP sockets are there?

✅ **Checkpoint:** You know your network environment and can diagnose connectivity.

🏁 Exercise 1.2: Key Network Configuration Commands

Objective: Practice common network configuration tasks.

Step 1: Interface management:

```
# Bring interface up/down (use your actual interface, e.g., eth0)
sudo ip link set eth0 down
sudo ip link set eth0 up

# Add an IP address
sudo ip addr add 192.168.100.1/24 dev eth0

# Remove an IP address
sudo ip addr del 192.168.100.1/24 dev eth0

# Change MTU
sudo ip link set eth0 mtu 9000    # Jumbo frames
sudo ip link set eth0 mtu 1500    # Standard
```

Step 2: Route management:

```
# Add a route
sudo ip route add 10.0.0.0/8 via 192.168.1.1

# Delete a route
sudo ip route del 10.0.0.0/8

# Add default gateway
sudo ip route add default via 192.168.1.1
```

Step 3: Network namespaces (isolated network stacks):

```
# Create a network namespace
sudo ip netns add testns

# Run commands in the namespace
sudo ip netns exec testns ip addr show

# Create veth pair and connect namespaces
sudo ip link add veth0 type veth peer name veth1
sudo ip link set veth1 netns testns

# Configure
sudo ip addr add 10.0.0.1/24 dev veth0
sudo ip link set veth0 up
sudo ip netns exec testns ip addr add 10.0.0.2/24 dev veth1
sudo ip netns exec testns ip link set veth1 up
sudo ip netns exec testns ip link set lo up

# Test connectivity
ping -c 2 10.0.0.2

# Cleanup
sudo ip netns del testns
```

✅ **Checkpoint:** You can configure interfaces, routes, and network namespaces.

LAB 2: Socket Fundamentals

🎯 Mission: "The First Connection"

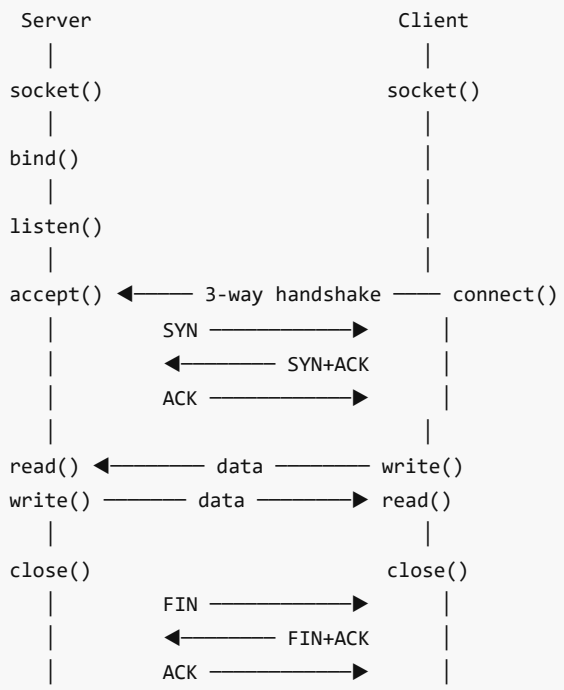
> ***Story:*** *Every network application — from web browsers to video streaming — starts with a single function call:* `socket()` *. This creates an endpoint for communication. Master the socket API, and you control the network.*

📖 Background

Socket = an endpoint for network communication, identified by:

- **Protocol family** (AF_INET for IPv4, AF_INET6 for IPv6, AF_UNIX for local)
- **Socket type** (SOCK_STREAM for TCP, SOCK_DGRAM for UDP, SOCK_RAW for raw)
- **Address** (IP address + port number)

The **TCP connection sequence**:



Address Structures and Casting

The socket API uses generic `struct sockaddr` that must be cast to protocol-specific structures:

```c
/* Generic (used in function prototypes) */
struct sockaddr {
    sa_family_t  sa_family;    /* Address family */
    char         sa_data[14];  /* Protocol-specific address */
};

/* IPv4 specific */
struct sockaddr_in {
    sa_family_t    sin_family;   /* AF_INET */
    in_port_t      sin_port;     /* Port (network byte order) */
    struct in_addr sin_addr;     /* IPv4 address */
};

/* IPv6 specific */
struct sockaddr_in6 {
    sa_family_t     sin6_family;   /* AF_INET6 */
    in_port_t       sin6_port;     /* Port */
    uint32_t        sin6_flowinfo; /* Flow info */
    struct in6_addr sin6_addr;     /* IPv6 address */
    uint32_t        sin6_scope_id; /* Scope ID */
};

/* Casting example: */
struct sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
inet_pton(AF_INET, "0.0.0.0", &addr.sin_addr);

/* Pass to socket functions with cast: */
bind(sockfd, (struct sockaddr *)&addr, sizeof(addr));
```

Byte order matters! Network byte order is big-endian:

```
htons(port)   /* Host TO Network Short (16-bit) */
htonl(addr)   /* Host TO Network Long (32-bit)  */
ntohs(port)   /* Network TO Host Short          */
ntohl(addr)   /* Network TO Host Long           */
```

🏁 Exercise 2.1: Your First TCP Socket

Objective: Create a basic TCP client-server pair.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab01_socket_basics/
cat socket_info.c
```

Step 2: Compile and run the socket info program:

```
make
./socket_info
```

This program demonstrates:

- Creating sockets with different types (STREAM, DGRAM, RAW)
- Address structure layout and sizes
- Byte order conversion functions
- `inet_pton()` and `inet_ntop()` for address conversion

Step 3: Study the byte order differences:

```
uint16_t port = 8080;
printf("Host order: 0x%04x\n", port);       /* 0x1f90 on little-endian */
printf("Network order: 0x%04x\n", htons(port)); /* 0x901f → big-endian */
```

Step 4: Answer in `answers.txt` :

1. What are the three arguments to `socket()` ?
2. Why is byte order conversion necessary?
3. What is the difference between `struct sockaddr` and `struct sockaddr_in` ?
4. What does `inet_pton()` do?

✅ **Checkpoint:** You understand socket creation and address structures.

🏁 Exercise 2.2: IPv6 Addressing

Objective: Understand IPv6 socket programming.

Step 1: IPv6 address formats:

```
Full:       2001:0db8:0000:0000:0000:0000:0000:0001
Compressed: 2001:db8::1
Loopback:   ::1
All zeros:  ::
IPv4-mapped: ::ffff:192.168.1.1
Link-local: fe80::1%eth0
```

Step 2: IPv6 socket creation:

```
/* IPv6 socket */
int sockfd = socket(AF_INET6, SOCK_STREAM, 0);

struct sockaddr_in6 addr;
memset(&addr, 0, sizeof(addr));
addr.sin6_family = AF_INET6;
addr.sin6_port = htons(8080);
inet_pton(AF_INET6, "::", &addr.sin6_addr);  /* All interfaces */

/* Dual-stack: accept both IPv4 and IPv6 */
int no = 0;
setsockopt(sockfd, IPPROTO_IPV6, IPV6_V6ONLY, &no, sizeof(no));
```

Step 3: Test IPv6 connectivity:

```
# Check IPv6 support
cat /proc/net/if_inet6

# Ping IPv6 loopback
ping6 -c 3 ::1

# IPv6 socket test
# Server: nc -6 -l 8080
# Client: nc -6 ::1 8080
```

Step 4: Examine the dual-stack server in the code template that accepts both IPv4 and IPv6.

✅ **Checkpoint:** You can create IPv6-capable socket programs.

LAB 3: TCP Client-Server Programming

🎯 Mission: "Building the Communication Bridge"

> ***Story:*** *Every web server, database, and chat application is built on TCP sockets. You will build a complete client-server system from scratch — handling connections, transferring data, and dealing with errors gracefully.*

📖 Background

TCP provides **reliable, ordered, byte-stream** delivery:

- Connection-oriented (3-way handshake)
- Guaranteed delivery (retransmissions)
- In-order data (sequence numbers)
- Flow control (sliding window)
- Congestion control (slow start, AIMD)

🏁 Exercise 3.1: Iterative TCP Server

Objective: Build a TCP server that handles one client at a time.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab02_tcp_server_client/
cat tcp_server.c
cat tcp_client.c
```

Step 2: Compile:

```
make
```

Step 3: Run the server:

```
./tcp_server 8080
```

Step 4: In another terminal, run the client:

```
./tcp_client 127.0.0.1 8080
```

Step 5: Study the server code — key system calls:

```c
/* 1. Create socket */
int server_fd = socket(AF_INET, SOCK_STREAM, 0);

/* 2. Set SO_REUSEADDR (avoid "Address already in use") */
int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

/* 3. Bind to address and port */
struct sockaddr_in addr = {
    .sin_family = AF_INET,
    .sin_port = htons(port),
    .sin_addr.s_addr = INADDR_ANY
};
bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));

/* 4. Listen for connections */
listen(server_fd, 5);  /* backlog = 5 */

/* 5. Accept a connection */
struct sockaddr_in client_addr;
socklen_t client_len = sizeof(client_addr);
int client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);

/* 6. Read/Write data */
char buf[1024];
ssize_t n = read(client_fd, buf, sizeof(buf));
write(client_fd, buf, n);

/* 7. Close */
close(client_fd);
close(server_fd);
```

Step 6: Answer in `answers.txt`:

1. What does `listen(fd, 5)` mean? What is the backlog?

2. What happens if you don't set `SO_REUSEADDR`?
3. Why does `accept()` return a NEW file descriptor?

✅ **Checkpoint:** You can build a basic TCP client-server pair.

🏁 Exercise 3.2: Concurrent TCP Server

Objective: Handle multiple clients simultaneously using fork() or threads.

Step 1: Examine the concurrent server:

```
cat tcp_server_concurrent.c
```

Step 2: This server uses `fork()` to handle each client:

```
while (1) {
    int client_fd = accept(server_fd, ...);
    pid_t pid = fork();
    if (pid == 0) {
        /* Child process: handle client */
        close(server_fd);
        handle_client(client_fd);
        close(client_fd);
        exit(0);
    }
    /* Parent: continue accepting */
    close(client_fd);
}
```

Step 3: Test with multiple simultaneous clients:

```
./tcp_server_concurrent 8080 &

# In multiple terminals:
./tcp_client 127.0.0.1 8080
./tcp_client 127.0.0.1 8080
./tcp_client 127.0.0.1 8080
```

Step 4: Observe the process tree:

```
ps aux | grep tcp_server
```

✅ **Checkpoint:** You can serve multiple clients concurrently.

🏁 Exercise 3.3: Case Study — Simple HTTP Server

Objective: Build a minimal HTTP/1.0 server to understand real-world TCP usage.

Step 1: Examine the HTTP server:

```
cat http_server.c
```

Step 2: Run and test:

```
./http_server 8080 &
curl -v http://127.0.0.1:8080/
curl http://127.0.0.1:8080/test.html
```

Step 3: Study the HTTP protocol over TCP:

```
Client → Server:
  GET /index.html HTTP/1.0\r\n
  Host: localhost\r\n
  \r\n

Server → Client:
  HTTP/1.0 200 OK\r\n
  Content-Type: text/html\r\n
  Content-Length: 45\r\n
  \r\n
  <html><body><h1>Hello!</h1></body></html>
```

Step 4: Capture with tcpdump:

```
sudo tcpdump -i lo -A port 8080
# In another terminal:
curl http://127.0.0.1:8080/
```

✅ **Checkpoint:** You understand how real protocols (HTTP) use TCP sockets.

LAB 4: IP, TCP, and UDP Headers

🎯 Mission: "Reading the Packet Blueprint"

> ***Story:*** *Every packet on the network carries headers — structured metadata that routers and hosts use to deliver data. Understanding these headers is the difference between writing code and understanding the network.*

📖 Background

IP Header (IPv4):

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|Version|  IHL  |Type of Service|          Total Length         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|         Identification        |Flags|    Fragment Offset      |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|  Time to Live |    Protocol   |       Header Checksum         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                       Source Address                          |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                    Destination Address                        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

TCP Header:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|          Source Port          |       Destination Port        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                        Sequence Number                        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                    Acknowledgment Number                      |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
| Data  |Reserv|C|E|U|A|P|R|S|F|                               |
|Offset |      |W|C|R|C|S|S|Y|I|          Window Size          |
|       |      |R|E|G|K|H|T|N|N|                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           Checksum            |        Urgent Pointer         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

UDP Header (much simpler):

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|          Source Port          |       Destination Port        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|            Length             |           Checksum            |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

🏁 Exercise 4.1: Packet Header Inspection

Objective: Parse and display IP, TCP, and UDP headers from raw packets.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab01_socket_basics/
cat packet_headers.c
```

Step 2: Compile and run:

```
make
sudo ./packet_headers
```

This program uses a raw socket to capture packets and dissects the IP, TCP, and UDP headers, printing each field.

Step 3: In another terminal, generate traffic:

```
ping -c 2 127.0.0.1
curl -s http://example.com > /dev/null
```

Step 4: Compare with tcpdump output:

```
sudo tcpdump -i any -nn -v -c 5
```

Step 5: Answer in `answers.txt`:

1. How many bytes is the minimum IP header?
2. What field in the IP header indicates TCP vs UDP?

3. How many bytes is the UDP header?
4. What are the SYN, ACK, FIN flags in the TCP header?

✅ **Checkpoint:** You can parse and understand network packet headers.

LAB 5: Socket Options

🎯 Mission: "Fine-Tuning the Connection"

> ***Story:*** *The default socket behavior is rarely optimal. Socket options let you control timeouts, buffer sizes, keepalives, and more. Knowing these options is the difference between a socket program that works and one that works well.*

📖 Background

Socket options are set with `setsockopt()` and read with `getsockopt()`:

```
int setsockopt(int sockfd, int level, int optname,
               const void *optval, socklen_t optlen);
int getsockopt(int sockfd, int level, int optname,
               void *optval, socklen_t *optlen);
```

Key levels: `SOL_SOCKET`, `IPPROTO_TCP`, `IPPROTO_IP`

🏁 Exercise 5.1: Exploring Socket Options

Objective: Read and set common socket options.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab01_socket_basics/
cat socket_options.c
```

Step 2: Compile and run:

```
make
./socket_options
```

Step 3: Study key socket options:

```
/* SO_REUSEADDR — allow reusing port immediately after close */
int opt = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

/* SO_RCVBUF / SO_SNDBUF — receive/send buffer sizes */
int bufsize = 65536;
setsockopt(fd, SOL_SOCKET, SO_RCVBUF, &bufsize, sizeof(bufsize));
setsockopt(fd, SOL_SOCKET, SO_SNDBUF, &bufsize, sizeof(bufsize));

/* SO_RCVTIMEO / SO_SNDTIMEO — read/write timeouts */
struct timeval tv = { .tv_sec = 5, .tv_usec = 0 };
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

/* SO_KEEPALIVE — detect dead connections */
int keepalive = 1;
```

```c
setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &keepalive, sizeof(keepalive));

/* TCP_NODELAY — disable Nagle algorithm (send immediately) */
int nodelay = 1;
setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &nodelay, sizeof(nodelay));

/* SO_LINGER — control close() behavior */
struct linger lg = { .l_onoff = 1, .l_linger = 5 };
setsockopt(fd, SOL_SOCKET, SO_LINGER, &lg, sizeof(lg));

/* IP_TTL — set IP time-to-live */
int ttl = 64;
setsockopt(fd, IPPROTO_IP, IP_TTL, &ttl, sizeof(ttl));

/* SO_BROADCAST — enable broadcast (UDP) */
int bcast = 1;
setsockopt(fd, SOL_SOCKET, SO_BROADCAST, &bcast, sizeof(bcast));
```

Step 4: Read current system defaults:

```bash
cat /proc/sys/net/core/rmem_default    # Default receive buffer
cat /proc/sys/net/core/rmem_max        # Max receive buffer
cat /proc/sys/net/core/wmem_default    # Default send buffer
cat /proc/sys/net/core/wmem_max        # Max send buffer
cat /proc/sys/net/ipv4/tcp_keepalive_time
cat /proc/sys/net/ipv4/tcp_keepalive_intvl
cat /proc/sys/net/ipv4/tcp_keepalive_probes
```

Step 5: Answer in `answers.txt`:

1. What does SO_REUSEADDR do and when is it needed?
2. What is the Nagle algorithm and when would you disable it?
3. What happens to pending data when you `close()` with SO_LINGER?

✅ **Checkpoint:** You can tune socket behavior with options.

LAB 6: UDP Programming

🎯 Mission: "The Speed Demon"

> ***Story:*** *TCP is reliable but has overhead — handshakes, acknowledgments, retransmissions. UDP fires packets into the network with no guarantees. It's used for DNS, video streaming, gaming, and IoT — anywhere speed matters more than guaranteed delivery.*

📖 Background

UDP is **connectionless** — no handshake, no state:

```
  Sender                        Receiver
    |                              |
socket()                       socket()
    |                              |
    |                          bind()
    |                              |
sendto() ——— UDP Datagram ———▶ recvfrom()
sendto() ——— UDP Datagram ———▶ recvfrom()
```

```
    |                           |
  close()                     close()
```

Key differences from TCP:

- **No connection** — each `sendto()` is independent
- **No reliability** — packets can be lost, duplicated, or reordered
- **Message boundaries** — each `sendto()` = one datagram (not a byte stream)
- **Faster** — no handshake, no ACK overhead

🏁 Exercise 6.1: UDP Echo Server

Objective: Build a UDP client-server pair.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab03_udp/
cat udp_server.c
cat udp_client.c
```

Step 2: Compile:

```
make
```

Step 3: Run server and client:

```
# Terminal 1: Server
./udp_server 9090

# Terminal 2: Client
./udp_client 127.0.0.1 9090
```

Step 4: Key UDP API calls:

```
/* Server: no listen() or accept() needed */
int fd = socket(AF_INET, SOCK_DGRAM, 0);
bind(fd, (struct sockaddr *)&addr, sizeof(addr));

/* Receive with sender address */
struct sockaddr_in client_addr;
socklen_t client_len = sizeof(client_addr);
ssize_t n = recvfrom(fd, buf, sizeof(buf), 0,
                     (struct sockaddr *)&client_addr, &client_len);

/* Reply to sender */
sendto(fd, buf, n, 0,
       (struct sockaddr *)&client_addr, client_len);
```

Step 5: Test message boundaries:

```
# Send multiple messages quickly
for i in 1 2 3 4 5; do
    echo "Message $i" | nc -u -q 0 127.0.0.1 9090
done
```

Step 6: Answer in `answers.txt`:

1. Why is there no `listen()` or `accept()` with UDP?
2. What happens if a UDP packet is lost?
3. Why does `recvfrom()` need the sender's address?

✅ **Checkpoint:** You can build UDP applications.

🏁 Exercise 6.2: UDP Broadcast

Objective: Send messages to all hosts on a network.

Step 1: Examine the broadcast example:

```
cat udp_broadcast.c
```

Step 2: Key concept — broadcasting:

```
/* Enable broadcast on socket */
int bcast = 1;
setsockopt(fd, SOL_SOCKET, SO_BROADCAST, &bcast, sizeof(bcast));

/* Send to broadcast address */
struct sockaddr_in bcast_addr = {
    .sin_family = AF_INET,
    .sin_port = htons(9090),
};
inet_pton(AF_INET, "255.255.255.255", &bcast_addr.sin_addr);
/* Or use subnet broadcast: 192.168.1.255 */

sendto(fd, msg, strlen(msg), 0,
       (struct sockaddr *)&bcast_addr, sizeof(bcast_addr));
```

Step 3: Test broadcast:

```
# Terminal 1: Listen for broadcasts
./udp_server 9090

# Terminal 2: Send broadcast
./udp_broadcast 9090 "Hello everyone!"
```

✅ **Checkpoint:** You understand UDP broadcast communication.

LAB 7: Name and Address Conversion

🎯 Mission: "The DNS Resolver"

> ***Story:*** *Users type "google.com", not "142.250.80.46". Your program must convert between human-readable names and numeric addresses. The `getaddrinfo()` function is your Swiss Army knife for address resolution.*

📖 Background

Modern address resolution uses `getaddrinfo()` — protocol-independent and thread-safe:

```
int getaddrinfo(const char *node,       /* hostname or IP string */
                const char *service,     /* port number or service name */
                const struct addrinfo *hints,  /* filter criteria */
                struct addrinfo **res);  /* result linked list */
```

Older (deprecated) functions: `gethostbyname()`, `gethostbyaddr()`, `inet_addr()`

🏁 Exercise 7.1: Address Resolution

Objective: Use `getaddrinfo()` for name resolution and build a DNS lookup tool.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab01_socket_basics/
cat dns_lookup.c
```

Step 2: Compile and test:

```
make
./dns_lookup example.com
./dns_lookup google.com
./dns_lookup ::1          # IPv6 loopback
```

Step 3: Key API usage:

```
struct addrinfo hints, *res, *p;
memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;      /* IPv4 or IPv6 */
hints.ai_socktype = SOCK_STREAM;  /* TCP */

int status = getaddrinfo("example.com", "80", &hints, &res);
if (status != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(status));
    return -1;
}

/* Walk the result list */
for (p = res; p != NULL; p = p->ai_next) {
    char ipstr[INET6_ADDRSTRLEN];
    void *addr;

    if (p->ai_family == AF_INET) {
        struct sockaddr_in *ipv4 = (struct sockaddr_in *)p->ai_addr;
        addr = &(ipv4->sin_addr);
    } else {
        struct sockaddr_in6 *ipv6 = (struct sockaddr_in6 *)p->ai_addr;
        addr = &(ipv6->sin6_addr);
    }

    inet_ntop(p->ai_family, addr, ipstr, sizeof(ipstr));
    printf("  %s\n", ipstr);
}

freeaddrinfo(res);  /* Don't forget to free! */
```

Step 4: Reverse lookup:

```
/* IP → hostname */
char host[NI_MAXHOST], service[NI_MAXSERV];
getnameinfo(p->ai_addr, p->ai_addrlen,
            host, sizeof(host),
            service, sizeof(service),
            0);
printf("Host: %s, Service: %s\n", host, service);
```

Step 5: Answer in `answers.txt`:

1. Why is `getaddrinfo()` preferred over `gethostbyname()`?
2. What does `AI_PASSIVE` do in hints?
3. Why must you call `freeaddrinfo()`?

✅ **Checkpoint:** You can resolve hostnames and addresses programmatically.

LAB 8: Advanced I/O

🎯 Mission: "The Multiplexer"

> ***Story:*** *A real server must handle hundreds of clients simultaneously without creating a thread or process for each one. The secret: I/O multiplexing. With `select()`, `poll()`, or `epoll()`, a single thread monitors multiple sockets and reacts only when data arrives.*

📖 Background

I/O multiplexing mechanisms:

```
┌──────────────┬────────────┬────────────┬──────────────────┐
│              │  select()  │  poll()    │  epoll           │
├──────────────┼────────────┼────────────┼──────────────────┤
│ Max FDs      │  1024      │ Unlimited  │ Unlimited        │
│ Scaling      │  O(n)      │ O(n)       │ O(1)             │
│ FD set copy  │  Every     │ Every      │ Once (kernel)    │
│              │  call      │ call       │                  │
│ Portability  │  POSIX     │ POSIX      │ Linux only       │
│ Best for     │  Small     │ Medium     │ Large scale      │
│              │  (< 100)   │            │ (1000s of FDs)   │
└──────────────┴────────────┴────────────┴──────────────────┘
```

🏁 Exercise 8.1: I/O Multiplexing with select/poll/epoll

Objective: Build a multiplexed server using each mechanism.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab06_advanced_io/
cat select_server.c
cat epoll_server.c
```

Step 2: Compile:

```
make
```

Step 3: Test the select-based server:

```
./select_server 8080 &

# Connect multiple clients
for i in 1 2 3 4 5; do
    echo "Hello from client $i" | nc -q 1 127.0.0.1 8080 &
done
wait
```

Step 4: Study select() usage:

```
fd_set readfds;
int maxfd = server_fd;

while (1) {
    FD_ZERO(&readfds);
    FD_SET(server_fd, &readfds);
    for (int i = 0; i < num_clients; i++)
        FD_SET(client_fds[i], &readfds);

    select(maxfd + 1, &readfds, NULL, NULL, NULL);

    if (FD_ISSET(server_fd, &readfds)) {
        /* New connection */
        int client = accept(server_fd, ...);
    }

    for (int i = 0; i < num_clients; i++) {
        if (FD_ISSET(client_fds[i], &readfds)) {
            /* Data from client i */
        }
    }
}
```

Step 5: Study epoll (Linux high-performance):

```
int epfd = epoll_create1(0);

struct epoll_event ev;
ev.events = EPOLLIN;
ev.data.fd = server_fd;
epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

struct epoll_event events[MAX_EVENTS];
while (1) {
    int n = epoll_wait(epfd, events, MAX_EVENTS, -1);
    for (int i = 0; i < n; i++) {
        if (events[i].data.fd == server_fd) {
            /* New connection */
        } else {
            /* Data from client */
        }
```

```
    }
}
```

Step 6: Non-blocking I/O:

```c
#include <fcntl.h>

/* Set socket to non-blocking */
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);

/* Now read() returns -1 with errno=EAGAIN when no data */
ssize_t n = read(fd, buf, sizeof(buf));
if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
    /* No data available — try later */
}
```

Step 7: Scatter-gather I/O:

```c
/* readv / writev — read/write to multiple buffers in one syscall */
struct iovec iov[2];
iov[0].iov_base = header;
iov[0].iov_len = header_len;
iov[1].iov_base = body;
iov[1].iov_len = body_len;

writev(fd, iov, 2);  /* Writes header + body atomically */
```

Step 8: Answer in `answers.txt`:

1. Why is epoll better than select for 10,000 connections?
2. What does non-blocking mode change about `read()`?
3. What is scatter-gather I/O used for?

✅ **Checkpoint:** You can build high-performance multiplexed servers.

LAB 9: Unix Domain Sockets

🎯 Mission: "The Local Express Lane"

> ***Story:*** *Not all communication crosses the network. When two processes on the same machine need to talk, Unix domain sockets provide a faster path — no TCP/IP overhead, no checksums, no routing. It's the local express lane.*

📖 Background

Unix domain sockets use the filesystem as the address:

```c
struct sockaddr_un {
    sa_family_t sun_family;    /* AF_UNIX */
    char sun_path[108];        /* Filesystem path */
};
```

Two types:

- `SOCK_STREAM` — byte stream (like TCP, but local)
- `SOCK_DGRAM` — datagrams (like UDP, but local and reliable)

🏁 Exercise 9.1: Unix Domain Socket Communication

Objective: Build a local IPC system using Unix domain sockets.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab05_domain_sockets/
cat unix_server.c
cat unix_client.c
```

Step 2: Compile and test:

```
make

# Terminal 1:
./unix_server

# Terminal 2:
./unix_client
```

Step 3: Key differences from TCP sockets:

```
/* Create Unix domain socket */
int fd = socket(AF_UNIX, SOCK_STREAM, 0);

/* Bind to a filesystem path */
struct sockaddr_un addr;
addr.sun_family = AF_UNIX;
strncpy(addr.sun_path, "/tmp/my_socket", sizeof(addr.sun_path) - 1);

/* Remove old socket file (important!) */
unlink("/tmp/my_socket");
bind(fd, (struct sockaddr *)&addr, sizeof(addr));
```

Step 4: File descriptor passing — Unix domain sockets can pass open file descriptors between processes:

```
/* Send a file descriptor using sendmsg() with SCM_RIGHTS */
struct msghdr msg = {0};
struct cmsghdr *cmsg;
char buf[CMSG_SPACE(sizeof(int))];

msg.msg_control = buf;
msg.msg_controllen = sizeof(buf);

cmsg = CMSG_FIRSTHDR(&msg);
cmsg->cmsg_level = SOL_SOCKET;
cmsg->cmsg_type = SCM_RIGHTS;
cmsg->cmsg_len = CMSG_LEN(sizeof(int));
*(int *)CMSG_DATA(cmsg) = fd_to_send;

sendmsg(sockfd, &msg, 0);
```

Step 5: Abstract sockets (Linux-specific — no filesystem entry):

```
/* Abstract namespace: first byte of sun_path is '\0' */
addr.sun_path[0] = '\0';
strncpy(addr.sun_path + 1, "my_abstract_socket", sizeof(addr.sun_path) - 2);
```

Step 6: Answer in `answers.txt` :

1. When would you use Unix domain sockets instead of TCP?
2. What is file descriptor passing used for?
3. What are abstract sockets?

✅ **Checkpoint:** You can use Unix domain sockets for local IPC.

LAB 10: Raw Sockets

🎯 Mission: "The Packet Crafter"

> ***Story:*** *Normal sockets hide the protocol headers from you. Raw sockets give you direct access to the IP layer — you craft your own headers, send custom packets, and sniff traffic. This is how network tools like* `ping` *,* `traceroute` *, and* `nmap` *work.*

📖 Background

Raw sockets operate below the transport layer:

```
Normal socket:   App ──▶ TCP/UDP ──▶ IP ──▶ Driver
                   (kernel handles headers)

Raw socket:      App ──▶ IP ──▶ Driver
                   (YOU craft TCP/UDP headers)

SOCK_RAW + IP_HDRINCL: App ──▶ Driver
                   (YOU craft IP header too)
```

> ⚠️ *Raw sockets require* `CAP_NET_RAW` *capability (usually root).*

🏁 Exercise 10.1: Packet Sniffer

Objective: Capture and decode packets using raw sockets.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab04_raw_sockets/
cat raw_sniffer.c
```

Step 2: Compile and run:

```
make
sudo ./raw_sniffer
```

Step 3: Study raw socket creation:

```
/* Capture all IP packets */
int fd = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
```

```
/* Or capture ALL protocols */
int fd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
```

Step 4: Generate traffic and observe:

```
# In another terminal:
ping -c 3 127.0.0.1
curl -s http://example.com > /dev/null
```

✅ **Checkpoint:** You can capture packets with raw sockets.

🏁 Exercise 10.2: ICMP Ping Implementation

Objective: Build your own `ping` tool using raw ICMP sockets.

Step 1: Examine the ping implementation:

```
cat ping_demo.c
```

Step 2: Compile and test:

```
make
sudo ./ping_demo 8.8.8.8
```

Step 3: Key ICMP ping logic:

```
/* Create raw ICMP socket */
int fd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);

/* Build ICMP Echo Request */
struct icmphdr icmp;
icmp.type = ICMP_ECHO;      /* 8 */
icmp.code = 0;
icmp.un.echo.id = htons(getpid());
icmp.un.echo.sequence = htons(seq++);
icmp.checksum = 0;
icmp.checksum = compute_checksum(&icmp, sizeof(icmp));

/* Send */
sendto(fd, &icmp, sizeof(icmp), 0, (struct sockaddr *)&dest, sizeof(dest));

/* Receive reply */
recvfrom(fd, buf, sizeof(buf), 0, NULL, NULL);
/* Parse: IP header (20 bytes) + ICMP reply */
```

Step 4: Answer in `answers.txt`:

1. Why do raw sockets require root?
2. What ICMP type/code is a ping request? A ping reply?
3. How is the ICMP checksum computed?

✅ **Checkpoint:** You understand raw socket programming and ICMP.

LAB 11: Datalink Access

🎯 Mission: "Layer 2 Control"

> ***Story:*** *Raw IP sockets operate at Layer 3. Sometimes you need to go deeper — to the Ethernet frame level. Datalink access lets you craft Ethernet frames, read MAC addresses, and implement protocols like ARP.*

📖 Background

Linux provides `AF_PACKET` sockets for Layer 2 access:

```
/* Capture all Ethernet frames */
int fd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));

/* Capture only ARP frames */
int fd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ARP));

/* SOCK_DGRAM: datalink header stripped */
int fd = socket(AF_PACKET, SOCK_DGRAM, htons(ETH_P_IP));
```

🏁 Exercise 11.1: Ethernet Frame Capture

Objective: Capture and decode Ethernet frames.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab04_raw_sockets/
cat ethernet_sniffer.c
```

Step 2: Compile and run:

```
make
sudo ./ethernet_sniffer
```

Step 3: The Ethernet frame structure:

```
struct ethhdr {
    unsigned char h_dest[6];    /* Destination MAC */
    unsigned char h_source[6];  /* Source MAC */
    __be16        h_proto;      /* Protocol (0x0800=IP, 0x0806=ARP) */
};
```

Step 4: Bind to a specific interface:

```
struct sockaddr_ll sll;
memset(&sll, 0, sizeof(sll));
sll.sll_family = AF_PACKET;
sll.sll_ifindex = if_nametoindex("eth0");
sll.sll_protocol = htons(ETH_P_ALL);
bind(fd, (struct sockaddr *)&sll, sizeof(sll));
```

✅ **Checkpoint:** You can access Layer 2 (Ethernet) frames.

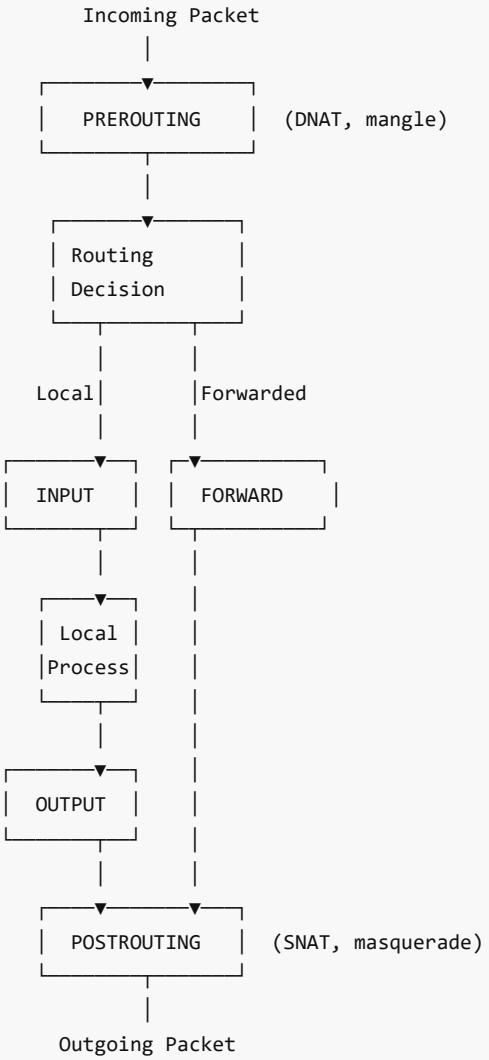
LAB 12: iptables, Routing, and Netfilter

🎯 Mission: "The Network Gatekeeper"

> ***Story:*** *Every Linux machine is a potential firewall and router.* `iptables` *controls which packets are allowed, blocked, or modified. Understanding the Netfilter framework — the kernel subsystem behind iptables — is essential for network security.*

📖 Background

Netfilter is the kernel's packet processing framework. `iptables` is the user-space tool to configure it:



🏁 Exercise 12.1: iptables Rules

Objective: Configure firewall rules using iptables.

Step 1: Explore current rules:

```
sudo iptables -L -n -v          # List all rules
sudo iptables -L -n -v -t nat   # NAT table
sudo iptables -L -n -v -t mangle # Mangle table
```

Step 2: Basic firewall rules:

```
# Block all incoming traffic on port 23 (Telnet)
sudo iptables -A INPUT -p tcp --dport 23 -j DROP

# Allow SSH (port 22)
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Allow established connections
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Block ICMP (ping)
sudo iptables -A INPUT -p icmp -j DROP

# Log dropped packets
sudo iptables -A INPUT -j LOG --log-prefix "DROPPED: "

# List rules with line numbers
sudo iptables -L -n --line-numbers

# Delete a rule by number
sudo iptables -D INPUT 3

# Flush all rules
sudo iptables -F
```

Step 3: NAT (Network Address Translation):

```
# Enable IP forwarding
sudo sysctl -w net.ipv4.ip_forward=1

# Masquerade (for internet sharing)
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

# Port forwarding (DNAT)
sudo iptables -t nat -A PREROUTING -p tcp --dport 8080 \
    -j DNAT --to-destination 192.168.1.100:80
```

Step 4: Routing:

```
# View routing table
ip route show
route -n

# Add static route
sudo ip route add 10.0.0.0/8 via 192.168.1.1

# Policy routing
sudo ip rule add from 192.168.1.0/24 table 100
sudo ip route add default via 192.168.2.1 table 100

# View routing cache
ip route show cache
```

✅ **Checkpoint:** You can configure iptables rules, NAT, and routing.

🏁 Exercise 12.2: Netfilter Queue — User-space Packet Processing

Objective: Write a C program that intercepts packets using NFQUEUE.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab07_netfilter_iptables/
cat nfqueue_demo.c
```

Step 2: Install dependencies:

```
sudo apt install -y libnetfilter-queue-dev
```

Step 3: Compile and run:

```
make

# Set up iptables to send packets to NFQUEUE
sudo iptables -A INPUT -p icmp -j NFQUEUE --queue-num 0

# Start the program
sudo ./nfqueue_demo

# In another terminal, ping to trigger:
ping -c 3 127.0.0.1

# Cleanup:
sudo iptables -D INPUT -p icmp -j NFQUEUE --queue-num 0
```

Step 4: The NFQUEUE program can:

- Inspect packet contents
- Accept (NF_ACCEPT) or drop (NF_DROP) packets
- Modify packets before sending

✅ **Checkpoint:** You can intercept and process packets from user space.

LAB 13: DHCP, DNS, and NTP

🎯 Mission: "The Infrastructure Protocols"

> ***Story:*** *Every device that connects to a network needs three things: an IP address (DHCP), a way to find servers by name (DNS), and accurate time (NTP). These infrastructure protocols are invisible when they work — and catastrophic when they fail.*

📖 Background

```
Device boots up:
  1. DHCP: "Give me an IP address" → gets 192.168.1.42
  2. DNS:  "Where is google.com?" → resolves to 142.250.80.46
  3. NTP:  "What time is it?" → synchronizes clock to UTC
```

🏁 Exercise 13.1: DHCP Protocol Analysis

Objective: Understand the DHCP exchange and build a DHCP packet parser.

Step 1: Observe a DHCP exchange:

```
# Capture DHCP traffic
sudo tcpdump -i eth0 -n port 67 or port 68 -v

# In another terminal, trigger DHCP:
sudo dhclient -v eth0   # Request new lease
```

Step 2: DHCP message exchange (DORA):

```
Client                        Server
  |                             |
  |──── DISCOVER (broadcast) ──▶|  "Any DHCP servers out there?"
  |                             |
  |◀──── OFFER ─────────────────|  "Here's 192.168.1.42"
  |                             |
  |──── REQUEST (broadcast) ───▶|  "I'll take 192.168.1.42"
  |                             |
  |◀──── ACK ───────────────────|  "It's yours for 24 hours"
```

Step 3: Examine current DHCP lease:

```
cat /var/lib/dhcp/dhclient.leases
# Or on systemd-networkd:
networkctl status
```

Step 4: Study the DHCP packet format in the code template.

✅ **Checkpoint:** You understand DHCP operation.

🏁 Exercise 13.2: DNS Protocol — Building a DNS Query

Objective: Build a DNS query from scratch using UDP sockets.

Step 1: Examine the DNS client:

```
cd code/network_programming/lab01_socket_basics/
cat dns_client.c
```

Step 2: DNS message format:

```
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                      ID                       |
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|QR|   Opcode  |AA|TC|RD|RA|   Z    |   RCODE    |
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                    QDCOUNT                    |
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                    ANCOUNT                    |
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

Step 3: Compile and test:

```
make
./dns_client example.com
```

```
./dns_client google.com
```

Step 4: Compare with system DNS tools:

```
dig example.com +short
dig example.com ANY
dig @8.8.8.8 example.com    # Use specific DNS server
```

✅ **Checkpoint:** You can construct and parse DNS messages.

🏁 Exercise 13.3: NTP Protocol

Objective: Understand NTP time synchronization.

Step 1: Check NTP status:

```
# systemd-timesyncd
timedatectl status
systemctl status systemd-timesyncd

# Traditional NTP
ntpq -p 2>/dev/null || chronyc sources 2>/dev/null
```

Step 2: NTP packet structure:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|LI | VN  |Mode |    Stratum    |     Poll      |   Precision   |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                       Root Delay                              |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                     Root Dispersion                           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                     Reference ID                              |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                   Reference Timestamp (64)                    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                   Originate Timestamp (64)                    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                    Receive Timestamp (64)                     |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                   Transmit Timestamp (64)                     |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

Step 3: Build and test the NTP client:

```
cat ntp_client.c
make
./ntp_client pool.ntp.org
```

Step 4: Analyze NTP traffic:

```
sudo tcpdump -i any port 123 -v
```

✅ **Checkpoint:** You understand NTP synchronization.

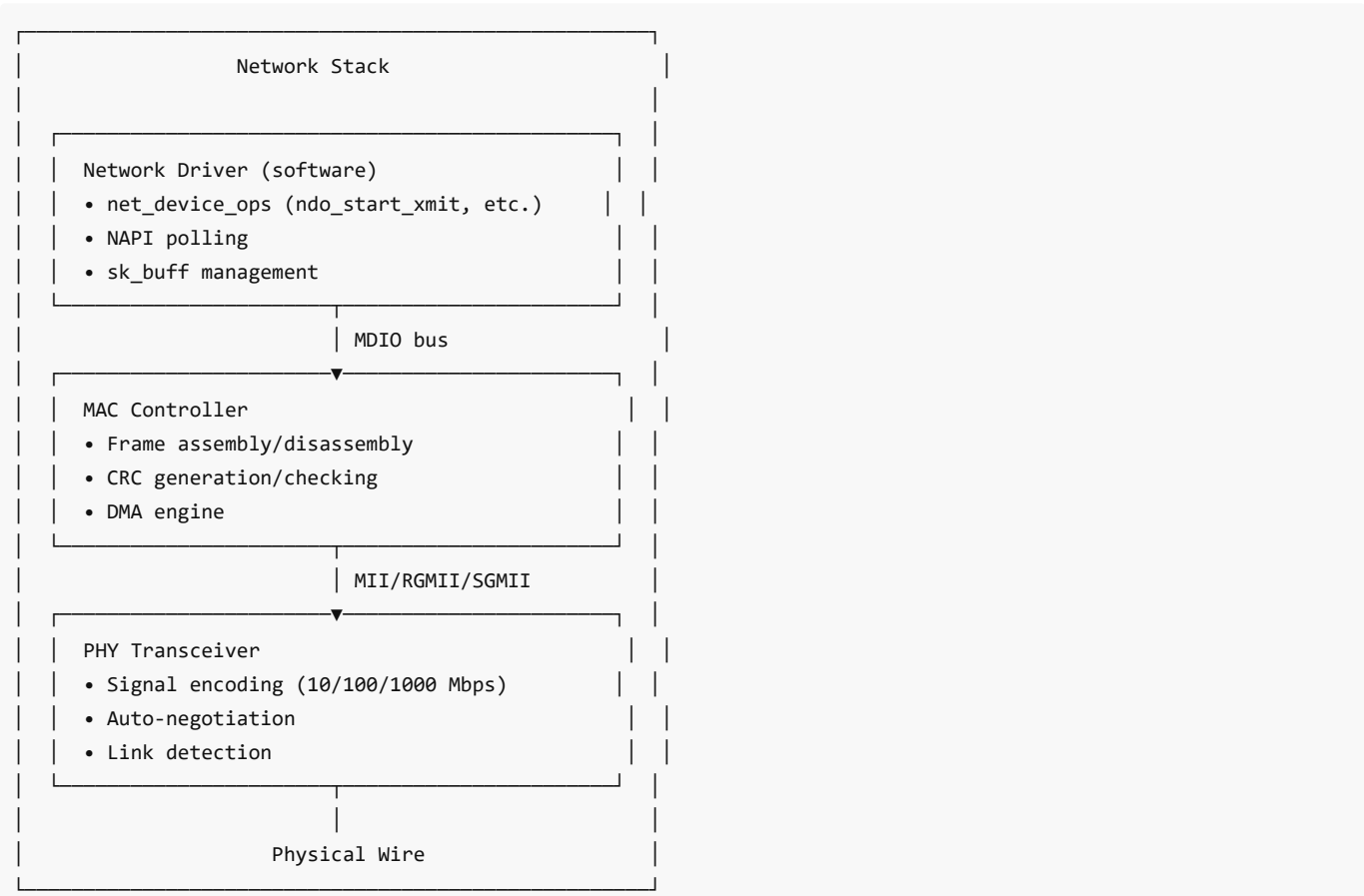
PART 2 — KERNEL NETWORK PROGRAMMING

LAB 14: Network Hardware — MAC and PHY

🎯 Mission: "The Hardware Foundation"

> ***Story:*** *Before packets reach software, they pass through hardware — the PHY (Physical Layer) transceiver that converts electrical/optical signals to bits, and the MAC (Media Access Control) that frames data for the wire. Understanding this boundary is where network drivers begin.*

📖 Background



🏁 Exercise 14.1: Exploring Network Hardware 🍓

Objective: Examine your system's network hardware through sysfs and ethtool.

Step 1: Explore network device sysfs:

```
# List network devices
ls /sys/class/net/

# Device details
cat /sys/class/net/eth0/address        # MAC address
cat /sys/class/net/eth0/mtu            # MTU
```

```
cat /sys/class/net/eth0/speed          # Link speed (Mbps)
cat /sys/class/net/eth0/duplex         # Full/half duplex
cat /sys/class/net/eth0/carrier        # Link state (1=up)
cat /sys/class/net/eth0/operstate      # State
cat /sys/class/net/eth0/type           # 1=Ethernet

# Driver info
ls -la /sys/class/net/eth0/device/driver
readlink /sys/class/net/eth0/device/driver
```

Step 2: Use ethtool:

```
sudo apt install -y ethtool

# Link settings
sudo ethtool eth0

# Driver info
sudo ethtool -i eth0

# Statistics
sudo ethtool -S eth0 | head -30

# PHY info
sudo ethtool -e eth0 2>/dev/null | head -20    # EEPROM dump

# Ring buffer sizes
sudo ethtool -g eth0

# Offload features
sudo ethtool -k eth0
```

Step 3: MDIO/PHY information:

```
# PHY devices
ls /sys/class/mdio_bus/ 2>/dev/null

# On Raspberry Pi:
ls /sys/class/net/eth0/phydev/ 2>/dev/null
cat /sys/class/net/eth0/phydev/phy_id 2>/dev/null
```

Step 4: Answer in `answers.txt` :

1. What driver is your network interface using?
2. What is your link speed and duplex setting?
3. What is the difference between MAC and PHY?

✅ **Checkpoint:** You understand the hardware beneath the network stack.

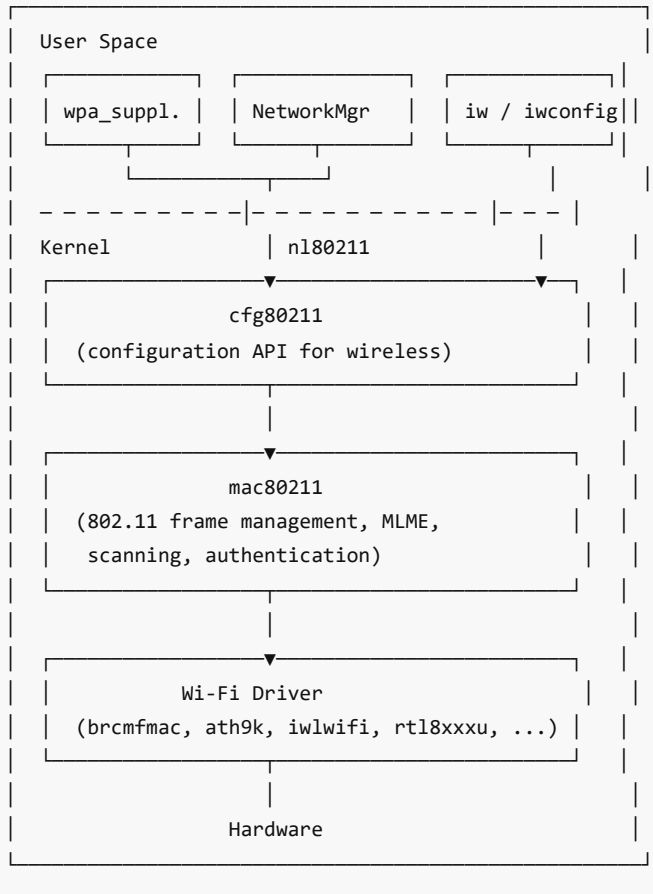
LAB 15: Network Drivers (Wi-Fi) 🍓

🎯 Mission: "The Wireless Explorer"

> ***Story:*** *Wi-Fi drivers are among the most complex in the kernel — they must handle radio frequency management, encryption, roaming, power saving, and the mac80211 subsystem. Understanding their structure reveals how the wireless world connects to the kernel.*

📖 Background

Linux wireless networking stack:



🏁 Exercise 15.1: Exploring Wireless Subsystem

Objective: Examine the wireless network subsystem and driver.

Step 1: Wireless interface discovery:

```
# List wireless interfaces
iw dev

# Interface info
iw dev wlan0 info

# Supported features
iw phy phy0 info | head -60

# Legacy tool
iwconfig wlan0
```

Step 2: Scan for networks:

```
# Trigger scan
sudo iw dev wlan0 scan | grep -E "SSID|signal|freq"
```

```
# Or using wpa_cli
wpa_cli scan
wpa_cli scan_results
```

Step 3: Wireless driver exploration:

```
# Which driver?
readlink /sys/class/net/wlan0/device/driver
lsmod | grep -i wifi
lsmod | grep -i brcm    # RPi
lsmod | grep -i iwl     # Intel

# Firmware
ls /lib/firmware/brcm/ 2>/dev/null    # RPi
dmesg | grep -i firmware | grep -i wlan

# Wireless statistics
cat /proc/net/wireless
iw dev wlan0 station dump
```

Step 4: mac80211 subsystem:

```
# mac80211 debug info
ls /sys/kernel/debug/ieee80211/ 2>/dev/null
cat /sys/kernel/debug/ieee80211/phy0/statistics/* 2>/dev/null

# Registered wireless phys
ls /sys/class/ieee80211/
```

Step 5: Answer in `answers.txt` :

1. What Wi-Fi driver is your system using?
2. What is mac80211's role?
3. What is cfg80211?

✅ **Checkpoint:** You understand the Linux wireless driver architecture.

LAB 16: Network Stack Analysis & Debugging

🎯 Mission: "Inside the Kernel Stack"

> ***Story:*** *When your application calls `send()` , the data passes through layers of kernel code — socket buffers, protocol handlers, routing, queueing disciplines, and finally the driver. Understanding this path is essential for diagnosing network performance issues.*

📖 Background

The kernel network stack data structure: **sk_buff** (socket buffer)

```
┌──────────────────────────────────────────────┐
│                  sk_buff                     │
│                                              │
│  head ──▶ ┌──────────────────┐               │
│           │  headroom        │               │
```

```
┌──────────────────────────────────────────────────────┐
│  data ──▶ ┌──────────────────────┐                   │
│           │  Ethernet header     │ (14 bytes)        │
│           ├──────────────────────┤                   │
│           │  IP header           │ (20 bytes)        │
│           ├──────────────────────┤                   │
│           │  TCP/UDP header      │ (20/8 bytes)      │
│           ├──────────────────────┤                   │
│           │  Payload             │                   │
│  tail ──▶ ├──────────────────────┤                   │
│           │  tailroom            │                   │
│  end ──▶  └──────────────────────┘                   │
│                                                      │
│  sk_buff metadata:                                   │
│    protocol, len, data_len, truesize,                │
│    dev (net_device), sk (socket),                    │
│    transport_header, network_header,                 │
│    mac_header                                        │
└──────────────────────────────────────────────────────┘
```

🏁 Exercise 16.1: Tracing the Network Stack

Objective: Trace a packet's journey through the kernel stack. 🍓

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab08_kernel_network/
cat explore_net_stack.sh
```

Step 2: Run the exploration script:

```
sudo ./explore_net_stack.sh
```

Step 3: Trace with ftrace:

```
# Trace key network functions
echo 'ip_rcv' > /sys/kernel/tracing/set_ftrace_filter
echo 'ip_local_deliver' >> /sys/kernel/tracing/set_ftrace_filter
echo 'tcp_v4_rcv' >> /sys/kernel/tracing/set_ftrace_filter
echo 'tcp_rcv_established' >> /sys/kernel/tracing/set_ftrace_filter
echo 'sock_recvmsg' >> /sys/kernel/tracing/set_ftrace_filter
echo function > /sys/kernel/tracing/current_tracer
echo 1 > /sys/kernel/tracing/tracing_on

# Generate traffic
curl -s http://example.com > /dev/null

echo 0 > /sys/kernel/tracing/tracing_on
cat /sys/kernel/tracing/trace | head -30
```

Step 4: Network stack sysctl tuning:

```
# TCP buffer sizes
cat /proc/sys/net/ipv4/tcp_rmem         # min default max
cat /proc/sys/net/ipv4/tcp_wmem
```

```
# Connection tracking
cat /proc/sys/net/netfilter/nf_conntrack_count
cat /proc/sys/net/netfilter/nf_conntrack_max

# TCP parameters
cat /proc/sys/net/ipv4/tcp_congestion_control
cat /proc/sys/net/ipv4/tcp_max_syn_backlog
cat /proc/sys/net/ipv4/tcp_fin_timeout
cat /proc/sys/net/ipv4/tcp_keepalive_time

# Network statistics
cat /proc/net/snmp | grep -i tcp
cat /proc/net/netstat | grep -i tcpext | head -2
```

Step 5: Use eBPF to trace network paths:

```
# Trace TCP connect with stack
sudo bpftrace -e '
kprobe:tcp_v4_connect {
    printf("connect: %s → %s\n", comm, kstack);
}'

# Trace packet receive path
sudo bpftrace -e '
kprobe:ip_rcv {
    @[kstack] = count();
}'
```

✅ **Checkpoint:** You can trace and analyze the kernel network stack.

LAB 17: ARP, ICMP, and IPv4 in the Kernel

🎯 Mission: "The Protocol Handlers"

> ***Story:*** *When a packet arrives, the kernel must decide which protocol handler processes it — ARP for address resolution, ICMP for control messages, TCP/UDP for transport. Understanding these handlers reveals how the kernel routes data.*

📖 Background

Protocol processing in the kernel:

```
Incoming packet → NIC driver → netif_receive_skb()
                                    │
                              ┌─────▼─────┐
                              │ ETH_P_IP  │──▶ ip_rcv()
                              │ ETH_P_ARP │──▶ arp_rcv()
                              │ ETH_P_IPV6│──▶ ipv6_rcv()
                              └───────────┘
                                    │
                              ┌─────▼─────┐
                              │IPPROTO_TCP│──▶ tcp_v4_rcv()
                              │IPPROTO_UDP│──▶ udp_rcv()
                              │IPPROTO_ICMP──▶ icmp_rcv()
                              └───────────┘
```

🏁 Exercise 17.1: ARP Protocol

Objective: Explore ARP (Address Resolution Protocol) operation.

Step 1: ARP cache:

```
# View ARP cache
ip neigh show
arp -a

# Clear ARP cache
sudo ip neigh flush all

# Watch ARP resolution
sudo tcpdump -i eth0 -n arp &
ping -c 1 <gateway_ip>
# You'll see: ARP Request (who-has) → ARP Reply (is-at)
```

Step 2: ARP in the kernel:

```
# ARP table size
cat /proc/sys/net/ipv4/neigh/default/gc_thresh1
cat /proc/sys/net/ipv4/neigh/default/gc_thresh2
cat /proc/sys/net/ipv4/neigh/default/gc_thresh3

# ARP timeout
cat /proc/sys/net/ipv4/neigh/default/base_reachable_time_ms
```

Step 3: Trace ARP in the kernel:

```
sudo bpftrace -e '
kprobe:arp_rcv {
    printf("ARP received on %s\n", comm);
}' 2>/dev/null &
sudo ip neigh flush all
ping -c 1 <gateway_ip>
```

✅ **Checkpoint:** You understand ARP resolution and kernel handling.

🏁 Exercise 17.2: ICMP Protocol

Objective: Explore ICMP handling in the kernel.

Step 1: ICMP types:

```
Type 0:  Echo Reply (ping response)
Type 3:  Destination Unreachable
Type 5:  Redirect
Type 8:  Echo Request (ping request)
Type 11: Time Exceeded (traceroute)
```

Step 2: ICMP in /proc:

```
# ICMP statistics
cat /proc/net/snmp | grep Icmp
```

```
# Rate limiting
cat /proc/sys/net/ipv4/icmp_ratelimit
cat /proc/sys/net/ipv4/icmp_ratemask
```

Step 3: Trace ICMP processing:

```
sudo bpftrace -e '
kprobe:icmp_rcv {
    printf("ICMP received: %s\n", comm);
}' &
ping -c 3 127.0.0.1
```

🏁 Exercise 17.3: IPv4 Processing

Objective: Explore IPv4 packet processing in the kernel.

Step 1: IPv4 kernel parameters:

```
# IP forwarding
cat /proc/sys/net/ipv4/ip_forward

# IP fragmentation
cat /proc/sys/net/ipv4/ipfrag_high_thresh
cat /proc/sys/net/ipv4/ipfrag_time

# IP statistics
cat /proc/net/snmp | grep "^Ip:"
```

Step 2: Trace IP receive path:

```
# Function graph of IP receive
echo function_graph > /sys/kernel/tracing/current_tracer
echo ip_rcv > /sys/kernel/tracing/set_graph_function
echo 1 > /sys/kernel/tracing/tracing_on
ping -c 1 127.0.0.1
echo 0 > /sys/kernel/tracing/tracing_on
cat /sys/kernel/tracing/trace | head -40
```

✅ **Checkpoint:** You understand how ARP, ICMP, and IP are processed in the kernel.

LAB 18: Adding a New Kernel Protocol

🎯 Mission: "Protocol Inventor"

> ***Story:*** *The kernel's protocol handler list isn't fixed. You can register your own protocol type that gets called when packets with your protocol number arrive. This is how new protocols (SCTP, DCCP, WireGuard) get added to Linux.*

📖 Background

The kernel protocol registration API:

```c
/* Register a protocol handler for a specific IP protocol number */
static struct net_protocol my_protocol = {
    .handler = my_protocol_rcv,
    .no_policy = 1,
};

/* Or register at the network device level */
static struct packet_type my_ptype = {
    .type = htons(ETH_P_MY_PROTO),
    .func = my_ptype_rcv,
};

/* In init: */
inet_add_protocol(&my_protocol, MY_PROTO_NUM);
/* or: */
dev_add_pack(&my_ptype);
```

🏁 Exercise 18.1: Custom Protocol Module

Objective: Write a kernel module that registers a custom protocol handler. 🍓

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab08_kernel_network/
cat custom_protocol.c
```

Step 2: Compile:

```
make
```

Step 3: Load and test:

```
sudo insmod custom_protocol.ko
dmesg | tail -5

# The module registers a handler for a custom EtherType
# Send a test packet from another terminal (requires raw socket)
sudo ./send_custom_packet

dmesg | tail -10    # See the received packet info

sudo rmmod custom_protocol
```

Step 4: Study the module:

```c
/* Protocol handler — called by kernel when matching packet arrives */
static int my_proto_rcv(struct sk_buff *skb, struct net_device *dev,
                        struct packet_type *pt, struct net_device *orig)
{
    /* skb contains the full packet */
    pr_info("Received %d bytes on %s\n", skb->len, dev->name);

    /* Parse the payload */
    unsigned char *data = skb->data;
```

```
    /* ... process data ... */

    kfree_skb(skb);
    return 0;
}
```

✅ **Checkpoint:** You can add custom protocol handlers to the kernel.

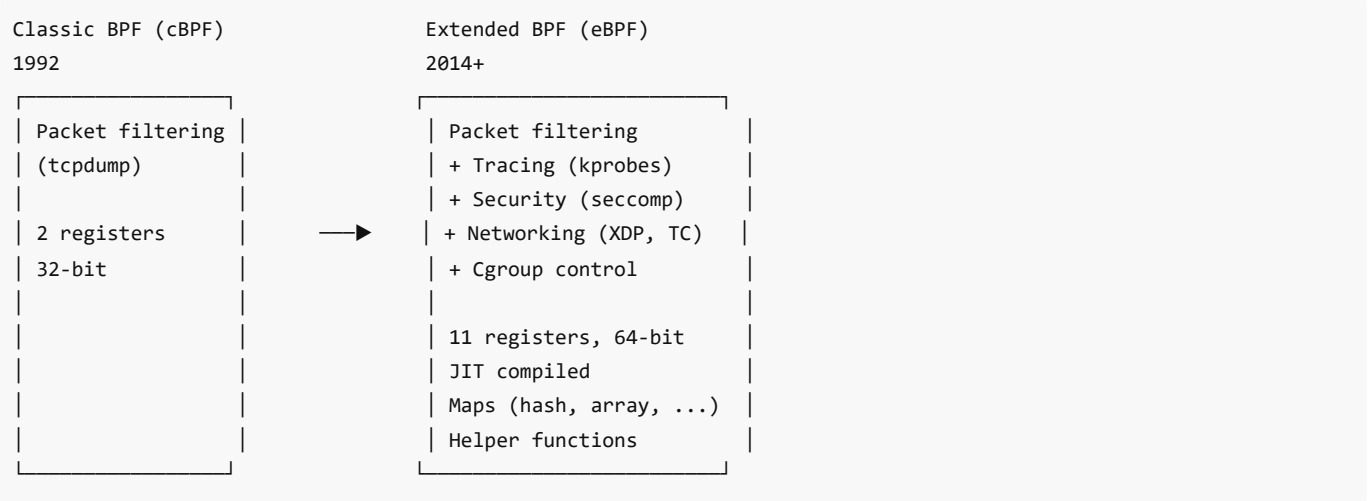
LAB 19: Berkeley Packet Filter (BPF/eBPF)

🎯 Mission: "The Programmable Kernel Filter"

> ***Story:*** *Classic BPF started as a packet filter for tcpdump. eBPF evolved it into a general-purpose in-kernel virtual machine. Now it's used for networking, security, tracing, and more. Understanding BPF is understanding the future of Linux networking.*

📖 Background

BPF evolution:



🏁 Exercise 19.1: Classic BPF — Socket Filtering

Objective: Use classic BPF to filter packets on a socket.

Step 1: Navigate to the lab directory:

```
cd code/network_programming/lab08_kernel_network/
cat bpf_filter.c
```

Step 2: Compile and run:

```
make
sudo ./bpf_filter
```

Step 3: Study the classic BPF filter:

```
/* BPF program to capture only ICMP packets */
struct sock_filter code[] = {
    /* Load EtherType field */
    BPF_STMT(BPF_LD | BPF_H | BPF_ABS, 12),
```

```
    /* Jump if not IP (0x0800) */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, ETH_P_IP, 0, 3),
    /* Load IP protocol field */
    BPF_STMT(BPF_LD | BPF_B | BPF_ABS, 23),
    /* Jump if ICMP (1) */
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, IPPROTO_ICMP, 0, 1),
    /* Accept */
    BPF_STMT(BPF_RET | BPF_K, 65535),
    /* Reject */
    BPF_STMT(BPF_RET | BPF_K, 0),
};

struct sock_fprog bpf = {
    .len = ARRAY_SIZE(code),
    .filter = code,
};

setsockopt(fd, SOL_SOCKET, SO_ATTACH_FILTER, &bpf, sizeof(bpf));
```

Step 4: Generate test traffic:

```
ping -c 3 127.0.0.1
curl -s http://example.com > /dev/null
# Only ICMP packets should be captured
```

✅ **Checkpoint:** You understand BPF socket filtering.

🏁 Exercise 19.2: eBPF with XDP

Objective: Understand XDP (eXpress Data Path) for high-performance packet processing.

Step 1: XDP concepts:

```
XDP runs at the earliest possible point — before sk_buff allocation:

Packet → NIC → Driver → XDP program → { XDP_PASS    → normal stack
                                      { XDP_DROP    → discard
                                      { XDP_TX      → send back out same NIC
                                      { XDP_REDIRECT → send to different NIC
```

Step 2: Explore XDP capabilities:

```
# Check XDP support
sudo bpftool feature probe | grep xdp

# List loaded XDP programs
sudo bpftool net list

# Simple XDP with bpftrace
sudo bpftrace -e '
kprobe:netif_receive_skb {
    @packets[((struct sk_buff *)arg0)->dev->name] = count();
}'
```

Step 3: XDP use cases:

- **DDoS mitigation** — drop attack packets before they reach the stack
- **Load balancing** — redirect packets to different NICs/CPUs
- **Monitoring** — count/classify packets at line rate
- **Firewalling** — packet filtering at driver level

Step 4: Answer in `answers.txt`:

1. What is the difference between cBPF and eBPF?
2. Where does XDP run in the network stack?
3. What are the 4 XDP return values?

✅ **Checkpoint:** You understand BPF filtering and XDP.

🏗️ MINI-PROJECT 1 — Multi-Protocol Chat System

> ***Objective:*** *Build a chat system that works over both TCP and UDP.*

Requirements:

1. **Server** that accepts both TCP and UDP clients (using `select()` or `epoll()`)
2. **TCP client** with reliable message delivery
3. **UDP client** with broadcast discovery
4. Message format with username, timestamp, and payload
5. Server broadcasts messages to all connected clients
6. Handle client disconnect gracefully

Deliverables:

- `chat_server.c` — Multi-protocol server
- `chat_client_tcp.c` — TCP chat client
- `chat_client_udp.c` — UDP chat client
- `protocol.h` — Message format definitions
- `Makefile`

🏗️ MINI-PROJECT 2 — Network Scanner

> ***Objective:*** *Build a network scanning tool using raw sockets.*

Requirements:

1. **ARP scanner** — discover hosts on the local subnet
2. **Port scanner** — TCP SYN scan using raw sockets
3. **Ping sweep** — ICMP-based host discovery
4. Output results in a formatted table
5. Support CIDR notation for target specification

Deliverables:

- `net_scanner.c` — Main scanner program
- `arp_scan.c` — ARP discovery module
- `port_scan.c` — Port scanner module
- `ping_sweep.c` — ICMP scanner module
- `Makefile`

🏗️ MINI-PROJECT 3 — Packet Capture and Analysis Tool

> ***Objective:*** *Build a simplified tcpdump clone with protocol dissection.*

Requirements:

1. Capture packets using raw sockets or libpcap
2. Dissect and display: Ethernet, IP, TCP, UDP, ICMP, ARP, DNS headers
3. Support BPF filter expressions
4. Save captures to pcap format
5. Display packet statistics (protocol counts, top talkers)
6. Support both live capture and file reading

Deliverables:

- `packet_cap.c` — Main capture program
- `dissect.c` — Protocol dissector module
- `pcap_io.c` — pcap file I/O
- `Makefile`

🧬 MINI-PROJECT 4 — Kernel Netfilter Module 🍓

> ***Objective:*** *Write a kernel module that implements a simple packet firewall using Netfilter hooks.*

Requirements:

1. Register NF_INET_PRE_ROUTING and NF_INET_LOCAL_OUT hooks
2. Accept/drop packets based on configurable rules
3. Rules configured via `/proc/my_firewall/rules`
4. Log dropped packets with source/destination
5. Statistics via `/proc/my_firewall/stats`

Deliverables:

- `my_firewall.c` — Netfilter kernel module
- `fw_control.sh` — Rule management script
- `Makefile`

🧬 MINI-PROJECT 5 — DNS Resolver Library

> ***Objective:*** *Build a DNS resolver library that constructs and parses DNS messages.*

Requirements:

1. Build DNS query packets from scratch (no libresolv)
2. Support A, AAAA, MX, CNAME, and TXT record types
3. Parse DNS response packets with pointer compression
4. Support multiple DNS servers with timeout and retry
5. Cache resolved entries with TTL expiration
6. Thread-safe API

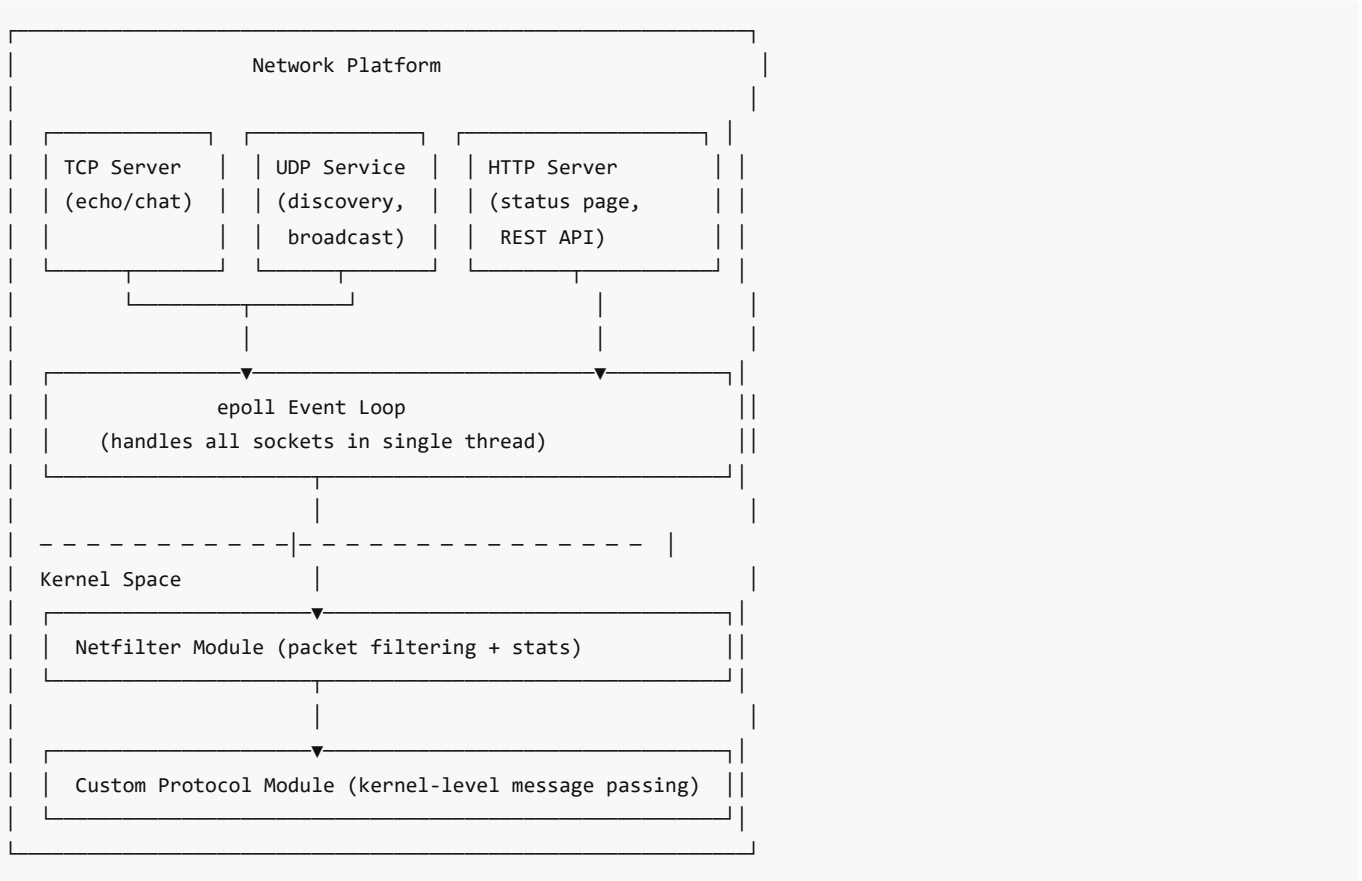
Deliverables:

- `dns_resolver.h` — Public API header
- `dns_resolver.c` — Implementation
- `dns_test.c` — Test program
- `Makefile`

🎓 CAPSTONE PROJECT — Multi-Service Network Platform

Overview

Build a comprehensive network platform that combines user-space socket programming with kernel-level networking. The system includes a multi-protocol server, custom kernel protocol module, and network monitoring capabilities.



Requirements

Phase 1: Core Server Platform

1. **Multi-protocol server** using epoll:
   - TCP echo/chat service on port 9000
   - UDP discovery service on port 9001
   - HTTP status/API on port 8080
2. Support 100+ concurrent TCP connections
3. Clean shutdown with signal handling

Phase 2: Protocol Implementation

4. Define a custom application protocol with:
   - Header (magic, version, type, length)
   - Message types: HELLO, DATA, STATUS, BYE
   - Serialization/deserialization functions
5. Build matching client

Phase 3: DNS & Service Discovery

6. Implement UDP-based service discovery (multicast or broadcast)
7. Build a simple DNS cache that resolves and caches names

8. NTP time synchronization client for log timestamps

Phase 4: Raw Socket Tools

9. Build an ICMP ping module for health checking
10. ARP cache manager for local network mapping
11. Packet capture module for debugging (pcap output)

Phase 5: Kernel Modules 🍓

12. **Netfilter firewall module**:
    - Configurable rules via procfs
    - Packet counting and logging
    - Rate limiting
13. **Custom protocol module**:
    - Register custom EtherType
    - Kernel-space packet handler
    - Communicate with user-space via netlink

Phase 6: Monitoring & Analysis

14. Network statistics dashboard (connection counts, bytes, errors)
15. Integration with eBPF probes for latency measurement
16. Comprehensive logging with syslog integration

Project Structure

```
capstone_network/
├── server/
│   ├── main.c              # epoll event loop
│   ├── tcp_service.c       # TCP echo/chat
│   ├── udp_service.c       # UDP discovery
│   ├── http_service.c      # HTTP status server
│   ├── protocol.h          # Custom protocol definitions
│   └── Makefile
├── client/
│   ├── client.c            # Multi-service client
│   ├── discovery.c         # Service discovery
│   └── Makefile
├── tools/
│   ├── ping_check.c        # ICMP health checker
│   ├── arp_scan.c          # ARP scanner
│   ├── dns_cache.c         # DNS resolver with cache
│   ├── ntp_sync.c          # NTP client
│   └── Makefile
├── kernel/
│   ├── nf_firewall.c       # Netfilter module
│   ├── custom_proto.c      # Custom protocol module
│   └── Makefile
├── monitor/
│   ├── net_stats.sh        # Statistics dashboard
│   └── ebpf_probes/        # bpftrace scripts
└── docs/
    ├── architecture.md      # System design document
    ├── protocol_spec.md     # Custom protocol specification
    └── test_plan.md         # Test procedures
```

Evaluation Criteria

- All services run concurrently from a single event loop
- Custom protocol is well-defined and handles errors
- Kernel modules load/unload cleanly
- System handles 100+ concurrent connections
- Comprehensive error handling throughout
- Network captures demonstrate correct protocol behavior
- Documentation covers architecture, protocol spec, and testing

---

End of Linux Network Programming — Hands-On Lab Manual